

Rapport Technique MSPR - ObRail Europe

- **Projet : ObRail Europe - Industrialisation et mise en production d'une plateforme ferroviaire européenne**
- **MSPR : TPRE532**
- **Promotion: B3**
- **Date : 18 mai 2026**
- **Document: Rapport technique de référence**
- **Version du document : v1.0**
- **Année : 2025-2026**
- **Statut : Version finale**
- **Encadrante pédagogique : *Dora Sabino***
- **Membres du groupe :**
 - **ABDILLAH I ABDI Mariam Marwo**
 - **SAMB Adja Nafissatou Lo**
 - **NKIBAN A ITCHIRI Orlane Emmanuelle Andrea**
 - **TOURE Zeinab Anne Marie**
 - **NDIAYE Mansour Djamil**

Suivi des tâches : Trello (lien fourni):

<https://trello.com/invite/b/69e74e583f650936f382ba17/ATTIaa05d72d0f16e3a2a3827bc407c678ffA9A7D7CE/mspr-b3>

Sommaire

1. Contexte et objectifs
2. Architecture globale de la solution (schemas)
3. Backend (FastAPI)
4. Frontend (React)
5. Base de données et modèle de données
6. ETL et data warehouse
7. Docker et orchestration
8. CI/CD
9. Stratégie de tests et résultats
10. Supervision, monitoring et logs
11. Sécurité
12. RGPD et accessibilité
13. Installation, exécution et exploitation
14. Maintenance, mises à jour et rollback
15. Organisation équipe, difficultés et résolutions
16. Axes d'amélioration et évolutions futures
17. Conclusion
18. Annexes

1. Contexte et objectifs

ObRail Europe est un observatoire de la mobilité ferroviaire européenne. Le projet MSPR vise à transformer un socle existant (MSPR précédent) en plateforme industrialisée, testée, monitorable et exploitable en conditions quasi-production.

1.1 Objectifs fonctionnels

- Centraliser des données multi-sources (Eurostat, GTFS, Back-on-Track, émissions CO2).
- Exposer des données métier via API REST documentée (/api/docs).
- Fournir deux interfaces React:
 - Interface externe (consultation publique et analyse).
 - Interface interne/admin (supervision, tests, opérations).
- Fournir analyses: timeline, classement CO2, comparaison jour/nuit, recommandations.

1.2 Objectifs non fonctionnels

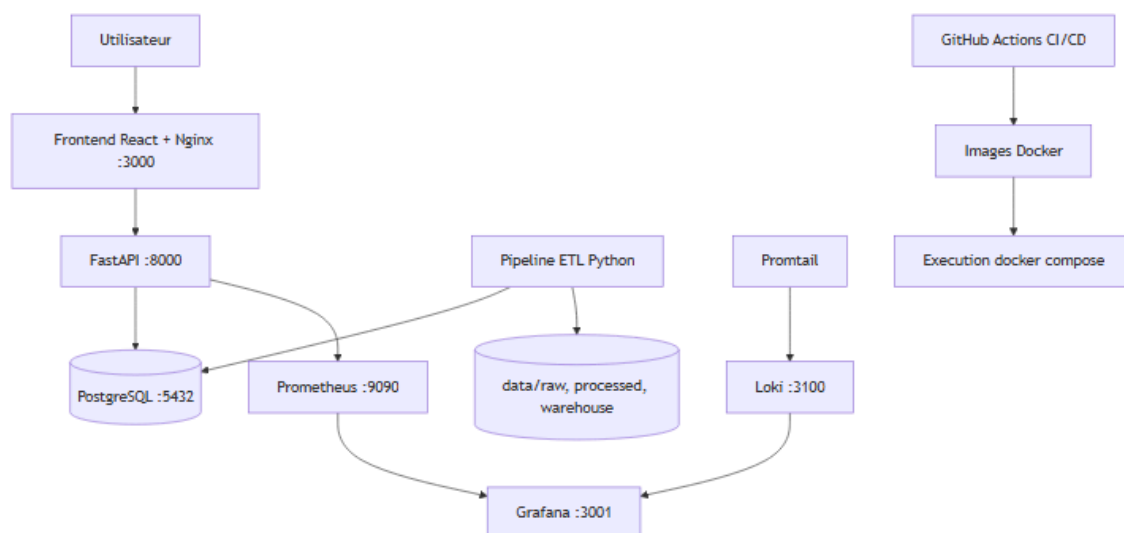
- Reproductibilité via Docker Compose.
- Qualité logicielle via CI/CD GitHub Actions.
- Observabilité via Prometheus + Grafana + Loki + Promtail.
- Maintenabilité via séparation claire des couches (ETL/API/Front/Monitoring).
- Conformité RGPD (pas de données personnelles, traçabilité, minimisation).
- Accessibilité (efforts RGAA/WCAG sur UI, navigation et lisibilité).

1.3 Conformité aux exigences MSPR

Exigence du sujet	Implementation realisee
Architecture globale documentee	✓ Schemas Mermaid et description complete
Pipeline CI/CD	✓ GitHub Actions
Strategie de tests	✓ Tests unitaires, integration, E2E
Supervision et monitoring	✓ Grafana, Prometheus, Loki, Promtail
Installation et execution	✓ Docker Compose documente
Securite	✓ Validation, isolation conteneurs, variables d'environnement
RGPD	✓ Minimisation des donnees et tracabilite
Accessibilite	✓ Efforts RGAA/WCAG
Maintenance et rollback	✓ Procedures documentees

2. Architecture globale de la solution (schemas)

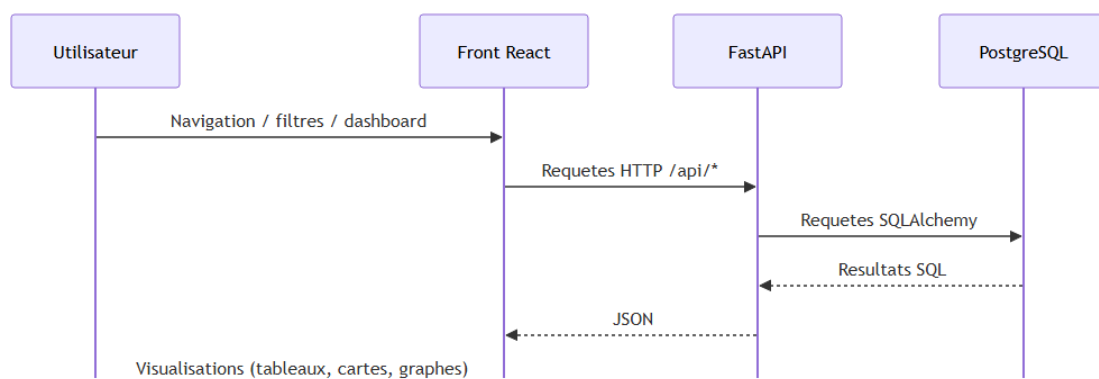
2.1 Schema d'architecture global



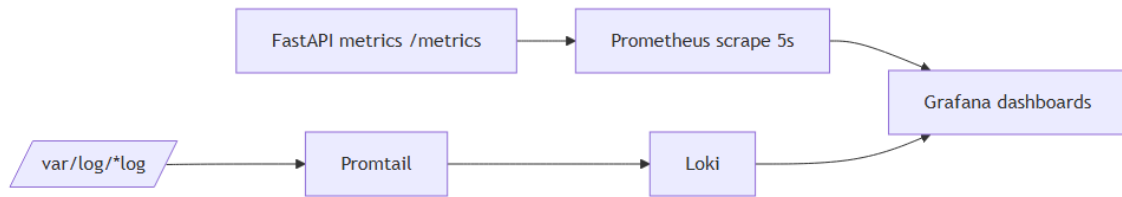
2.1.1 Arborescence générale du projet

```
project/  
├─ platform/  
│   ├── server/  
│   └─ front/  
├─ etl/  
├─ monitoring/  
│   ├── grafana/  
│   ├── prometheus/  
│   ├── loki/  
│   └─ promtail/  
├─ sql/  
├─ docs/  
├─ .github/workflows/  
└─ docker-compose.yml
```

2.2 Flux applicatifs



2.3 Architecture monitoring



2.4 Principaux choix techniques (argumentés)

Technologie	Alternatives possibles	Justification du choix
FastAPI	Flask, Django	Documentation OpenAPI automatique, performances async, integration simple avec tests et monitoring
React	Vue.js, Angular	Ecosysteme mature, modularite forte, integration simple avec bibliotheques graphiques
PostgreSQL	MySQL, MariaDB	Robustesse relationnelle, vues SQL avancees, compatibilite analytique
Docker Compose	Kubernetes	Solution adaptee a la taille du projet MSPR et plus simple a maintenir
GitHub Actions	GitLab CI, Jenkins	Integration native GitHub et configuration rapide
Grafana	Kibana	Simplicite de configuration et visualisation temps reel
Playwright	Cypress	Compatibilite multi-navigateurs et automatisation moderne

3. Backend (FastAPI)

3.1 Structure backend

- Dossier principal: **platform/server/app**
- Fichiers centraux:
 - **main.py**: creation app, CORS, routeurs, healthcheck, docs /api/docs.
 - **database.py**: connexion SQLAlchemy via variables d'environnement.
 - **models.py**: dimensions, faits, vues.
 - **routers/*.py**: logique API par domaine métier.

3.2 Endpoints exposés

Principaux endpoints valides et utilises:

- **Pays et stats:**
 - GET /api/countries
 - GET /api/countries/stats
- **Trains:**
 - GET /api/night-trains
 - GET /api/night-trains/night
 - GET /api/night-trains/day
 - GET /api/night-trains/summary
 - GET /api/night-trains/by-operator/{operator_id}
- **Dashboard:**
 - GET /api/dashboard/metrics
 - GET /api/dashboard/kpis
- **Analyses:**
 - GET /api/statistics/timeline
 - GET /api/statistics/co2-ranking
 - GET /api/analysis/train-types-comparison
 - GET /api/analysis/policy-recommendations
- **Operateurs:**
 - GET /api/operators
 - GET /api/operators/{operator_id}/stats
- **Geographie:**
 - GET /api/geographic/coverage
- **Metadonnees:**
 - GET /api/metadata/quality
 - GET /api/metadata/sources
- **Interne/admin:**
 - GET /api/internal/overview
 - POST /api/internal/diagnostic/run

- POST /api/internal/tests/run
- **Sante:**
 - GET /health

3.3 Points techniques importants

- Suppression de la limite fixe 500 sur certains flux de trains: la pagination supporte limit=None (retour non borne si necessaire).
- CORS configure pour http://localhost:3000.
- Instrumentation Prometheus active via prometheus-fastapi-instrumentator.
- Logique de supervision interne: API appelle Prometheus/Grafana, lit rapports JSON ETL, execute diagnostic/tests.

3.4 Performance et optimisation

Plusieurs mécanismes d'optimisation ont été mis en place afin de garantir des performances acceptables sur des volumes importants de données:

- Suppression de la limitation fixe de 500 trains afin de permettre une exploitation complète des données.
- Mise en place de plafonds de rendu front (MAX_TRAINS_TOTAL, MAX_TRAINS_PER_TYPE) afin de limiter la surcharge navigateur.
- Utilisation du clustering Leaflet (leaflet.markercluster) pour réduire la densité visuelle et le cout de rendu.
- Utilisation de vues SQL (dashboard_metrics, operator_dashboard) afin d'éviter des calculs lourds répétés cote API.
- Séparation ETL/API afin d'éviter les traitements intensifs en temps réel.

4. Frontend (React)

4.1 Architecture front

- Dossier: **platform/front/app/src**

- **Routing principal:**
 - **Externe:** /externe/HomePage, /externe/Trajets, /externe/Map, /externe/Statistique, /externe/Operateur
 - **Interne:** /interne/HomePage
- Services API via Axios dans src/services/api.js.

4.2 Interface externe

- Consultation des trains (jour/nuit), stats, operateurs.
- Carte Leaflet avec clustering pour lisibilité sur forte densité.
- Graphiques Chart.js (barres, comparatifs, KPIs).

4.3 Interface interne (admin)

- Onglets Dashboard / Monitoring / Logs / Tests & Qualité / CI-CD.
- Rafraichissement périodique configurable (10s, 30s, 60s).
- Lancement depuis UI de:
 - Diagnostic ETL
 - Tests backend
- Affichage de l'état des services docker détectés localement.

4.4 Point clé de conception map (problème densité)

Problème rencontre: surcharge visuelle de la carte due au volume de trains.

Solution appliquée: usage de `leaflet.markercluster` + plafonds de rendu (`MAX_TRAINS_TOTAL`, `MAX_TRAINS_PER_TYPE`) afin d'améliorer UX et performances.

5. Base de données et modèle de données

5.1 Schema relationnel (SQL)

Script de référence : `sql/01_init.sql`

- **Dimensions:**
 - dim_countries
 - dim_years
 - dim_operators
 - dim_stops
- **Faits:**
 - facts_night_trains (jour+nuite, distance, duree)
 - facts_country_stats (passagers, emissions, CO2/passager)
- **Vues:**
 - dashboard_metrics
 - operator_dashboard

5.2 Contraintes et intégrité

- Clés primaires sur toutes dimensions/faits.
- Clés étrangères des faits vers dimensions.
- Vues d'agrégation pour simplifier front et API.

5.3 MCD/MPD

Les schemas de modélisation de l'itération précédente sont conservés dans:

- archive/mspr n1/docs/mcd.png
- archive/mspr n1/docs/mpd.png

Ils servent de base conceptuelle à l'évolution MSPR actuelle.

6. ETL et data warehouse

6.1 Pipeline ETL

- Point d'entrée : **etl/main_etl.py** et **etl/run_full_etl.py**

- Phases:
 - i. Extraction (etl/extract/*)
 - ii. Transformation (etl/transform/*)
 - iii. Chargement (etl/load/*)

6.2 Sources traitées

- GTFS France
- GTFS Suisse
- GTFS Allemagne
- Eurostat trafic/passagers
- Eurostat émissions CO2
- Back-on-Track (trains de nuit)

6.3 Audit qualité ETL

- Diagnostic: **etl/audit/diagnostic.py**
- Rapport qualité API: **platform/server/app/reports/quality_reports.json**
- Endpoint expose: **GET /api/metadata/quality**

6.4 Pourquoi ce design ETL

- Modularité par source et par étape (extract/transform/load).
- Réexécution partielle possible (debug plus rapide).
- Traite des sources hétérogènes sans couplage fort au backend.

7. Docker et orchestration

7.1 Services docker-compose

Fichier: **docker-compose.yml**

Services principaux:

- **front** (React build + Nginx)
- **api** (FastAPI)
- **db** (PostgreSQL)
- **etl** (pipeline complet)
- **prometheus**
- **grafana**
- **loki**
- **promtail**

7.2 Volumes et persistance

- Persistance BDD: volume db_data.
- Provisioning grafana versionne dans monitoring/grafana/provisioning.
- Dashboards exportes en JSON
(monitoring/grafana/dashboards/obrail-dashboard.json) pour éviter leur perte au redémarrage.

7.3 Ports

- **Front:** http://localhost:3000
- **API:** http://localhost:8000
- **Swagger:** http://localhost:8000/api/docs
- **Prometheus:** http://localhost:9090
- **Grafana:** http://localhost:3001
- **Loki:** http://localhost:3100
- **PostgreSQL:** localhost:5432

8. CI/CD

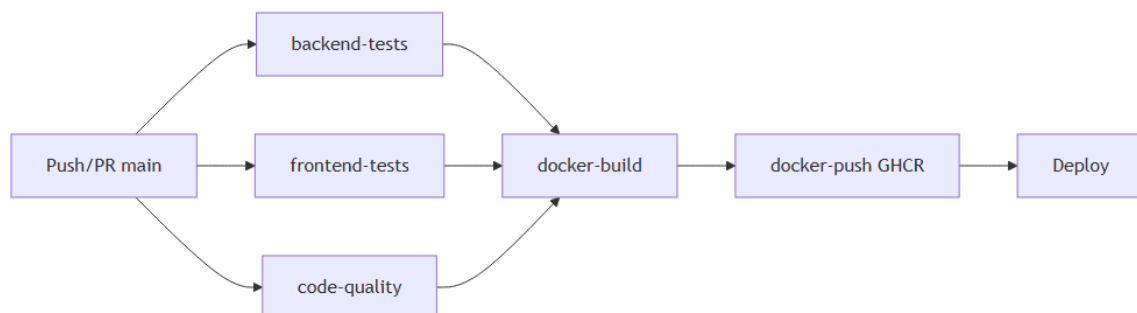
8.1 Workflow GitHub Actions

Fichier: .github/workflows/ci-cd.yml

Jobs identifies:

- **backend-tests**
 - Pytest unitaires
 - Pytest integration
 - Pytest E2E API
- **frontend-tests**
 - Installation Node + Playwright
 - Démarrage docker local dans job
 - E2E Playwright
- **code-quality**
 - Flake8
 - ESLint
 - Prettier
- **docker-build** (si tests passent)
- **docker-push vers GHCR** (sur push main)
- **deploy**

8.2 Pipeline simplifié



8.3 Point bloquant CI/CD sur interface admin

Le front interne affiche un état CI/CD local (état conteneurs) via **PipelinesTab**.
Limite actuelle: absence d'integration API GitHub Actions (historique de runs non expose en natif dans l'UI).

Décision projet: fonctionnalité potentielle, abandon possible si indisponible dans les délais.

9. Stratégie de tests et résultats

9.1 Typologie

- **Backend unitaires** : platform/server/test/unit/*
- **Backend integration** : platform/server/test/integration/api_db/*
- **Backend E2E API** : platform/server/test/E2E/*
- **ETL unitaires** : etl/test/test_transformations.py
- **Front E2E Playwright** : platform/front/app/tests/e2e/*

9.1.1 Volume des tests

Type de test	Description
Tests unitaires backend	Validation logique metier et fonctions isolees
Tests integration	Validation interactions API/Base de donnees
Tests E2E API	Validation comportement complet endpoints
Tests ETL	Verification transformations et qualite donnees
Tests E2E front	Validation parcours utilisateur React

Les tests sont exécutés automatiquement dans le pipeline CI/CD GitHub Actions.

9.2 Exemples vérifiés

- Validations endpoint pays/stats/train/operators.
- Vérifications géographiques, metadata, dashboard.

- Parcours utilisateur front externe et interne.
- Endpoints internes **/api/internal/*** testés en E2E front.

9.3 Résultat global

- Le pipeline CI est conçu pour bloquer build/push si les tests principaux échouent.
- Les tests de qualité (lint/format) sont informatifs (**continue-on-error**) et peuvent être renforcés plus tard.

10. Supervision, monitoring et logs

10.1 Stack mise en place

- Prometheus: scrape toutes les 5 secondes (**monitoring/prometheus/prometheus.yml**).
- Grafana: datasource provisionnée automatiquement + dashboard charge depuis fichier JSON.
- Loki + Promtail: centralisation des logs système.

Le monitoring applicatif repose sur une architecture d'observabilité centralisée:

- Prometheus collecte les métriques exposées par FastAPI via **/metrics**.
- Grafana permet la visualisation temps réel des indicateurs critiques.
- Loki centralise les logs applicatifs et système.
- Promtail collecte automatiquement les logs des conteneurs Docker.

Cette architecture permet:

- le suivi des performances,
- la détection d'erreurs,
- l'analyse des logs,
- et le diagnostic rapide des incidents.

10.2 Problème rencontre et résolution

Problème: dashboards Grafana perdus après redémarrage des conteneurs.

Solution: création/configuration UI puis export JSON versionne (monitoring/grafana/dashboards/obrail-dashboard.json) + provisioning automatique.

10.3 Données observables

- Sante API (up)
- Débit requêtes
- Erreurs 5xx
- Latence moyenne et p95
- Etats des services docker (via endpoint interne)
- Logs centralises via Loki

11.Sécurité

11.1 Mesures en place

- Validation des params via FastAPI/Pydantic (types, bornes **Query**).
- CORS explicite vers front local.
- Variables d'environnement pour configuration DB/services.
- Séparation des rôles de services par conteneur.
- Tests automatiques régulièrement exécutés.

11.2 Limites identifiées

- Valeurs sensibles en dur dans **docker-compose.yml** (mot de passe DB, JWT secret) à corriger via **.env** prive + secrets CI/CD.
- Pas d'authentification forte exposée côté API publique dans l'état actuel.

11.3 Recommandations

- Introduire gestion de secrets centralisée.
- Ajouter authentification/autorisation (JWT + rôles admin/public).

- Introduire rate limiting et audit sécurité OWASP API.

11.4 Bonnes pratiques appliquées

Le projet suit plusieurs recommandations inspirées des bonnes pratiques OWASP:

- Validation stricte des entrées utilisateur.
- Isolation des services par conteneur Docker.
- Limitation de l'exposition des services internes.
- Séparation claire des couches applicatives.
- Limitation des données sensibles exposées.

12. RGPD et accessibilité

12.1 RGPD

Mesures appliquées:

- Données manipulées majoritairement non personnelles (transport/statistiques agrégées).
- Principe de minimisation: pas de collecte d'identifiants utilisateur nominatif.
- Transparence: endpoints metadata (**/api/metadata/sources**, **/api/metadata/quality**).
- Traçabilité ETL via rapports d'audit.

Points de vigilance:

- Si futur ajout d'authentification/utilisateurs: registre de traitements, durées de conservation, droits d'accès/suppression à formaliser.

12.2 Accessibilité (RGAA/WCAG)

Travail déclaré et implémenté sur le front :

- Navigation structurée interne/externe.

- Labels et structuration des blocs critiques.
- Lisibilité des pages et adaptation responsive.
- Effort spécifique map dense via clustering pour limiter surcharge cognitive/visuelle.

Améliorations conseillées :

- Audit outille (axe-core, Lighthouse, RGAA checklist) avec score versionne.
- Revue contraste/couleurs et parcours clavier complet documente.

13. Installation, exécution et exploitation

13.1 Prérequis

- Docker
- Docker Compose
- (optionnel dev front) Node.js 20+
- (optionnel dev API) Python 3.11+

13.2 Lancement standard

```
docker compose up --build
```

13.3 Arrêt

```
docker compose down
```

13.4 Logs

```
docker compose logs -f
```

13.5 Lancement E2E front

```
cd platform/front/app
npm ci
npm run e2e:install
npm run e2e
```

14. Maintenance, mises à jour et rollback

14.1 Maintenance courante

- Mise à jour dépendances backend/front avec validation CI complète.
- Régénération ETL planifiée et vérification des rapports qualité.
- Monitoring continu des erreurs/latences via Grafana.

14.2 Correctifs

- Branches de correction ciblées, PR, exécution pipeline, merge main.
- Vérification post-correctif: endpoints critiques + smoke tests UI.

14.3 Stratégie rollback

- Rollback code: retour commit Git stable précédent.
- Rollback images: redéploiement tag GHCR stable precedent.
- Rollback data: restauration volume DB depuis sauvegarde SQL si nécessaire.

Procédure type :

1. Identifier version stable (commit/tag image).
2. Redéployer image stable (**docker compose pull** puis **up -d**).
3. Verifier **health**, endpoints critiques, dashboard monitoring.
4. Documenter incident et RCA (Root Cause Analysis).

15. Organisation équipe, difficultés et résolutions

15.1 Répartition des rôles

- **NDIAYE Mansour Djamil :**
 - Reprise Docker/conteneurs et architecture du code.
 - Ajout de données warehouse (trajets).
 - Mise en place monitoring (Grafana, Prometheus, Loki, Promtail).
 - Support Andrea sur finalisation tests E2E backend (contrainte de temps).
- **NKIBAN A ITCHIRI Orlane Emmanuelle Andrea :**
 - Evolution API (nouveaux endpoints).
 - Corrections backend (dont limite de récupération trains retirée).
 - Partie principale des tests E2E.
- **SAMB Adja Nafissatou Lo :**
 - Tests unitaires, integration et pipeline CI/CD.
 - Gestion de l'implémentation et de l'orchestration des tests en pipeline.
- **ABDILLAH ABDI Mariam Marwo :**
 - Front externe client, liaison API, respect RGAA.
 - Interface interne orientée graphes/visualisation.
 - Résolution problème de densité de la carte via clustering Leaflet.
- **TOURE Zeinab Anne Marie :**
 - Interface admin/interne (avec appui Djamil sur certaines liaisons).
 - Intégration de composants de supervision et flux internes.

15.2 Difficultés majeures et solutions

- Densité carte trop forte:
 - Solution: marker clustering + limites de rendu.
- Dashboards Grafana perdus au restart:
 - Solution: export JSON + provisioning versionne.

- Découverte Playwright et couverture multi-front:
 - Solution: progressive setup + documentation docs/E2E_TESTS.md + integration CI.
- Affichage CI/CD dans front admin:
 - Solution transitoire: état local Docker; évolution prévue via API GitHub.

Difficultés rencontrées par SAMB Adja Nafissatou Lo

La principale difficulté concernait l'intégration des tests dans le pipeline CI/CD, notamment les tests E2E.

Le lancement automatique des services Docker et l'orchestration des dépendances nécessaires aux tests ont demandé plusieurs ajustements du workflow GitHub Actions.

Solutions apportées :

- Ajout d'étapes de démarrage automatique Docker Compose dans les jobs CI.
- Stabilisation des temps d'attente et de disponibilité des services.
- Séparation des tests unitaires et E2E afin de simplifier le debug.

Difficultés rencontrées par ABDILLAH ABDI Mariam Marwo

Le principal problème concernait l'affichage de la carte interactive due au volume très important de trains affichés simultanément.

Solutions apportées :

- Intégration de leaflet.markercluster.
- Mise en place de plafonds de rendu afin d'améliorer les performances et la lisibilité.
- Travail complémentaire sur les règles RGAA/WCAG et la lisibilité des interfaces.

Difficultés rencontrées par TOURE Zeinab Anne Marie

La principale difficulté concernait l'intégration de la supervision et de certaines données CI/CD dans l'interface admin React.

Solutions apportées :

- Mise en place d'une supervision locale des services Docker.
- Liaison entre endpoints internes FastAPI et composants React admin.
- Etude d'une future integration GitHub Actions API.

Difficultés rencontrées par NKIBAN A ITCHIRI Orlane Emmanuelle Andrea

Les principales difficultés ont concerné:

- la suppression des limitations historiques de certains endpoints,
- et l'apprentissage de Playwright pour les tests E2E.

Solutions apportées :

- Refactorisation des endpoints critiques.
- Mise en place progressive des tests automatisés.
- Documentation technique dans docs/E2E_tests.md.

Difficultés rencontrées par NDIAYE Mansour Djamil

Les principales difficultés ont concerné:

- la restructuration Docker,
- l'industrialisation du monitoring,
- et la persistance des dashboards Grafana.

Solutions apportées :

- Réorganisation des conteneurs et volumes Docker.
- Mise en place du provisioning Grafana.
- Export JSON versionné des dashboards afin d'éviter leur perte après redémarrage.

15.3 Approche collective

L'équipe a fait circuler les connaissances entre membres pour que chacun comprenne les sujets transverses (API, front, tests, infra), ce qui a renforcé l'autonomie globale.

16. Axes d'amélioration et évolutions futures

16.1 Priorités à court terme

- Externaliser toutes les valeurs sensibles dans `.env/secrets`.
- Ajouter mesure de coverage automatisée avec seuil bloquant.
- Connecter un véritable statut GitHub Actions dans l'interface admin.

16.2 Evolutions techniques

- Authentification JWT + rôles (public/admin).
- Caching API pour endpoints lourds.
- Alerting Grafana/Prometheus plus fin (seuils, notifications).
- Sauvegarde/restoration DB automatisée.

16.3 Evolution métier majeure proposée

Ajout d'un système de prédiction :

- Prédiction de demande passagers par pays/période.
- Estimation des émissions futures selon scenarios de trafic.
- Priorisation de politiques ferroviaires (recommandations data-driven).

Piste technique : modèle de prévision temporelle (Prophet/XGBoost/LSTM selon maturité), expose via endpoints API dédiés.

16.4 Limites actuelles du projet

Malgré l'industrialisation réalisée, certaines limites techniques restent identifiées:

- Absence d'authentification forte sur l'API publique.
- Absence d'orchestration Kubernetes.
- Couverture de tests non encore imposée par seuil bloquant.
- Monitoring principalement local et non distribué.
- Absence actuelle de système d'alerting automatisé (mail, webhook).

- Intégration GitHub Actions incomplète dans l'interface admin.

16.5 Perspectives MLOps et intelligence artificielle

Le projet pourra évoluer vers une architecture orientée MLOps afin d'intégrer des fonctionnalités prédictives avancées.

Pistes envisagées:

- entraînement automatisé de modèles de prédiction,
- monitoring des performances des modèles IA,
- versionning des modèles,
- pipeline de traitement automatisé des données d'apprentissage,
- endpoints FastAPI dédiés aux prédictions.

Des modèles de séries temporelles (Prophet, XGBoost, LSTM) pourront être testés afin d'anticiper:

- la demande passagers,
- les émissions CO2,
- les besoins d'infrastructures ferroviaires.

17. Conclusion

Le projet MSPR ObRail Europe a permis de mettre en œuvre une démarche d'industrialisation complète autour d'une plateforme de traitement et d'analyse de données ferroviaires européennes.

Au-delà des fonctionnalités métier, le projet a intégré des dimensions essentielles de l'ingénierie logicielle moderne:

- *conteneurisation,*
- *CI/CD,*
- *supervision,*
- *tests automatisés,*
- *sécurité,*
- *observabilité,*
- *maintenabilité,*
- *et préparation à l'évolution future vers des usages IA et prédictifs.*

Le socle réalisé constitue une base technique robuste, évolutive et exploitable dans un contexte proche des contraintes professionnelles réelles.

18. Annexes

18.1 Lien vers le GitHub

MSPR_1_B3

18.2 Fichiers de référence

- **API docs:** <http://localhost:8000/api/docs>
- **CI/CD:** `.github/workflows/ci-cd.yml`
- **Compose:** `docker-compose.yml`
- **SQL init:** `sql/01_init.sql`
- **ETL:** `etl/main_etl.py`, `etl/run_full_etl.py`
- **Monitoring:**
 - `monitoring/prometheus/prometheus.yml`
 - `monitoring/grafana/provisioning/*`
 - `monitoring/grafana/dashboards/obrail-dashboard.json`
 - `monitoring/loki/local-config.yaml`
 - `monitoring/promtail/config.yml`
- **Tests:**
 - Backend: `platform/server/test/*`
 - ETL: `etl/test/test_transformations.py`
 - Front E2E: `platform/front/app/tests/e2e/*`
- **Historique MSPR N1:**
 - `archive/mspr n1/docs/rapport_technique.md`

18.3 Commandes utiles

```
# Demarrer
docker compose up --build

# Voir l'etat
docker compose ps

# Logs
docker compose logs -f

# Arrêter
docker compose down

# Tests backend locaux (exemple)
python -m pytest -v platform/server/test/unit

# Tests front E2E
cd platform/front/app
npm run e2e
```